



.txt



.csv



.json

Archivos de Texto, CSV y JSON en Python

Leer, escribir y gestionar archivos desde tu programa

Skill: Python I

Docente: Jhonathan Arley Peñaloza Sierra

Campuslands · 2026

Objetivos de Aprendizaje

1 Comprender qué son los archivos de texto y para qué sirven

2 Conocer la función `open()` y sus modos de apertura

3 Leer, escribir y añadir contenido a archivos con `with`

4 Trabajar con archivos CSV usando el módulo `csv`

5 Manipular archivos JSON con el módulo `json`

¿Qué son los Archivos de Texto?



Piensa en un cuaderno...

Un cuaderno:

- Guarda texto escrito a mano
- Puedes leerlo cuando quieras
- Puedes agregar más páginas
- Puedes borrar y reescribir

Es información que perdura.



Un archivo de texto es igual...

Un archivo de texto:

- Guarda caracteres legibles
- Se lee con un programa
- Se puede agregar contenido
- Se puede sobrescribir

Los datos persisten en disco.



¿Por qué son importantes?

Sin archivos, los datos se pierden cuando el programa termina. Los archivos permiten guardar información de forma permanente. En Python usamos `open()` para trabajar con ellos.

Tipos de archivos: .txt (texto plano) • .csv (datos tabulares) • .json (datos estructurados)

La función open()

open() es la función de Python para abrir archivos. Recibe el nombre del archivo y el modo.

```
open(file, mode='r') ← file = nombre, mode = modo de apertura
```

Los 4 modos principales:

"r" Leer

Solo lectura. El archivo debe existir.

"w" Escribir

Solo escritura. BORRA lo anterior. Crea si no existe.

"a" Añadir

Agrega al final. NO borra. Crea si no existe.

"r+" Leer+Escribir

Lectura y escritura. El archivo debe existir.

Tabla de modos de open()

Modo	Operación	Si existe	Si NO existe
"r"	Solo lectura	Abre	Error
"w"	Solo escritura	Borra contenido	Crea
"a"	Añadir al final	Agrega al final	Crea
"r+"	Leer + escribir	Abre	Error
"w+"	Escribir + leer	Borra contenido	Crea
"a+"	Añadir + leer	Agrega al final	Crea
"rb"	Leer binario	Abre	Error
"wb"	Escribir binario	Borra contenido	Crea



Importante:

"w" BORRA todo el contenido anterior. Si quieres agregar sin borrar, usa "a".

Modo Lectura: "r"

Leer un archivo es la operación más básica. Dos formas de hacerlo:

Forma tradicional:

```
file = open("archivo.txt", "r")
print(file.read())
file.close()
# ¡No olvides cerrar el archivo!
```

⚠ Debes cerrar con `.close()`

✓ Con with (recomendado):

```
with open("archivo.txt") as f:
    print(f.read())

# Se cierra automáticamente
```

✓ Se cierra solo al terminar

💡 ¿Por qué usar with?

with cierra el archivo automáticamente al salir del bloque, incluso si hay un error. Es la forma recomendada.

Modo Escritura "w" y Añadir "a"



Modo "w" — Escribir

BORRA todo y escribe de nuevo.

Si no existe, lo crea.

```
with open("archivo.txt","w") as f:
    f.write("Hola mundo")
# Contenido anterior: BORRADO
```



Modo "a" — Añadir

CONSERVA todo y agrega al final.

Si no existe, lo crea.

```
with open("archivo.txt","a") as f:
    f.write("\nOtra linea")
# Se agrega AL FINAL
```

Otros modos:

```
# "r+" — Leer y escribir en el mismo archivo
with open("archivo.txt", "r+") as f:
    contenido = f.read()      # Lee primero
    f.write("\nOtra linea")  # Escribe al final

# "rb" — Leer archivos binarios (imágenes, PDFs)
with open("imagen.png", "rb") as f: contenido = f.read()
```

La cláusula with: Buenas Prácticas

✗ Sin with (riesgoso):

```
file = open("archivo.txt", "r")
contenido = file.read()
print(contenido)
file.close() # ¡Si olvidas esto,
             # el archivo queda
             # abierto!
```

*Si hay error antes de .close(),
el archivo nunca se cierra.*

✓ Con with (recomendado):

```
with open("archivo.txt") as f:
    contenido = f.read()
    print(contenido)

# Al salir del bloque with,
# el archivo se cierra
# automáticamente. ¡Siempre!
```

*Aunque haya error, el archivo
se cierra correctamente.*

📌 Regla de oro: SIEMPRE usa with open() para trabajar con archivos. Es más seguro y limpio.



Actividad: Archivos de Texto

1. Crea un archivo "mi_diario.txt" con modo "w" y escribe tu nombre y la fecha
2. Cierra y vuelve a abrir con "r" para leer e imprimir su contenido
3. Abre con modo "a" y agrega: "Hoy aprendí archivos en Python"
4. Lee de nuevo el archivo completo e imprímelo
5. Repite todos los pasos usando with en lugar de open/close



Tiempo: 10 minutos

Archivos CSV: Datos Tabulares

CSV = Comma Separated Values (Valores Separados por Comas)

Es el formato más usado para importar y exportar datos de bases de datos y hojas de cálculo.

The image shows a CSV file named 'Tabla Monitores: Bloc de notas' in a text editor. The file contains a single line of text with semicolons as separators. This data is then imported into an Excel spreadsheet, where the semicolons are converted into commas, creating a table with 6 columns and 6 rows of data.

	A	B	C	D	E	F
1		X25	XB323QK NV	XB273U GX	XB253Q GZ	XB273U GS
2	Pantalla	24.5"	31.5"	27"	24.5"	27"
3	Resolucion	FHD	4K	QHD	FHD	QHD
4	Panel	IPS	IPS	IPS	IPS	IPS
5	HDR	No	HDR400	HDR400	No	HDR400
6						
7						
8						
9						
10						
11						

Usos comunes: Importar/exportar clientes • Productos • Órdenes • Reportes analíticos

💡 Un archivo .csv es texto plano donde cada línea es una fila y los valores se separan por comas (o punto y coma).

CSV: Leer con reader()

El módulo csv de Python tiene funciones para leer y escribir archivos CSV.

 Archivo a.csv:

```
1,2,3,4
2
3
4,Estudio de python
```

```
from csv import reader

with open("a.csv", "r") as file:
    lector = reader(file)
    for row in lector:
        print(row)
```

Salida: ['1','2','3','4'] ['2'] ['3'] ['4','Estudio de python']

CSV: Escribir con writer()

```
from csv import writer

myData = [
    ["nombre","apellido","nota"],
    ["Alex","Godoy","2"],
    ["Carlos","De la Torre","2"]]

with open("personas.csv","w",newline="") as f:
    escritor = writer(f)
    escritor.writerows(myData)
```

CSV: DictReader (leer como diccionarios)

DictReader convierte cada fila del CSV en un diccionario, usando la primera fila como llaves.

 ventas_semana_pasada.csv: DIA,JORNADA1,JORNADA2,JORNADA3 / LUNES,340,360,417 / MARTES,318,342,391 ...

```
import csv

with open('ventas_semana_pasada.csv') as f:
    lector = csv.DictReader(f)
    for fila in lector:
        print(fila['DIA'], fila['JORNADA1'])
```

¿Qué pasa?

Cada fila se convierte en un diccionario automáticamente:

```
{'DIA': 'LUNES',
 'JORNADA1': '340',
 'JORNADA2': '360',
 'JORNADA3': '417'}
```



Ventaja de DictReader:

Accedes a los datos por nombre de columna (fila['DIA']) en lugar de por índice (fila[0]). Mucho más legible.

CSV: DictWriter (escribir desde diccionarios)

DictWriter escribe diccionarios a un archivo CSV. Necesitas definir los fieldnames (nombres de columnas).

```
from csv import DictWriter

headers = ["nombre", "edad", "ciudad"]
datos = [
    {"nombre": "Ana", "edad": 25, "ciudad": "Bogotá"},
    {"nombre": "Luis", "edad": 30, "ciudad": "Medellín"}
]

with open("personas.csv", "w", newline="") as f:
    writer = DictWriter(f, fieldnames=headers)
    writer.writeheader()    # Escribe los encabezados
    for persona in datos:
        writer.writerow(persona)
```

Para importar CSV en Excel:

Datos → Obtener datos → De un archivo → De texto/CSV



Archivos JSON: Datos Estructurados

JSON = JavaScript Object Notation — Formato universal para transmitir datos entre sistemas.

Formato de JSON:

{ }

Delimitan objetos

[]

Denotan arrays/listas

:

Separa clave y valor

,

Separa elementos

Ejemplos:

Texto simple:

```
{ "pais": "España" }
```

Array:

```
{ "ciudades": ["Barcelona",  
               "Madrid"] }
```

Objeto:

```
{ "ciudad": { "nombre": "Valencia",  
              "codigo": "VLC" } }
```

Lista de objetos:

```
{  
  "ciudades": [  
    { "nombre": "Barcelona",  
      "codigo": "BCN" },  
    { "nombre": "Madrid",  
      "codigo": "MAD" },  
    { "nombre": "Valencia",  
      "codigo": "VLC" }  
  ]  
}
```

JSON: Leer con json.load()

Para leer un archivo JSON y convertirlo a un diccionario de Python:

 **object.json:**

```
[
  {"name": "John",
   "age": 30,
   "car": null},
  {"name": "Javier",
   "age": 33,
   "car": "Mazda 3"}
]
```

```
import json

with open("object.json") as file:
    users = json.load(file)

for user in users:
    print("Nombre:", user["name"],
          "Edad:", user["age"])
```

Salida: Nombre: John Edad: 30 / Nombre: Javier Edad: 33



json.load() vs json.loads():

json.load(file) lee desde un ARCHIVO. json.loads(string) lee desde un STRING. No confundirlos.

JSON: Escribir con json.dumps()

Para guardar un diccionario de Python como archivo JSON:

```
import json

diccionario = [
    {"table": 12, "chair": 14, "id": 567},
    {"table": 44, "chair": 10, "id": 56}
]

# Convertir diccionario a texto JSON
things = json.dumps(diccionario)

# Guardar en archivo
with open("things.json", "w") as file:
    file.write(things)
```

¿Qué hace?

1. Crea el diccionario en Python
2. json.dumps() lo convierte a texto JSON
3. Abre archivo con "w"
4. Escribe el texto JSON
5. El archivo se cierra solo (with)

Archivo generado:

```
things.json →
[{"table":12,...},...]
```



json.dumps() vs json.dump():

json.dumps() devuelve un STRING (luego lo escribes con write). json.dump(data, file) escribe directo al archivo.

Comparación: .txt vs .csv vs .json

	.txt	.csv	.json
¿Qué es?	Texto plano libre	Tabla con separadores	Datos estructurados
Estructura	Sin estructura fija	Filas y columnas	Objetos clave:valor
Leer con	<code>open() + read()</code>	<code>csv.reader/DictReader</code>	<code>json.load()</code>
Escribir con	<code>open() + write()</code>	<code>csv.writer/DictWriter</code>	<code>json.dumps()</code>
Uso ideal	Notas, logs, config	Datos tabulares, Excel	APIs, config, datos web

 **Regla simple:**
.txt para texto libre. .csv cuando tienes datos en tabla. .json cuando tienes datos con estructura clave-valor.



Actividad: CSV y JSON

1. Crea un archivo "estudiantes.csv" con `writer()` que tenga: nombre, edad, nota
2. Agrega al menos 3 estudiantes al CSV
3. Lee el CSV con `DictReader` e imprime cada estudiante como diccionario
4. Crea un archivo "productos.json" con `json.dumps()` con 3 productos (nombre, precio)
5. Lee el JSON con `json.load()` e imprime cada producto
6. Calcula el precio total de todos los productos del JSON



Tiempo: 15 minutos

Errores Comunes

`FileNotFoundError`

Causa: Leer archivo inexistente con "r"

Solución: Verifica la ruta o usa "w"/"a" para crear

`Usar "w" sin querer`

Causa: "w" borra todo el contenido anterior

Solución: Si quieres agregar, usa "a"

`json.decoder.JSONDecodeError`

Causa: El archivo JSON tiene formato inválido

Solución: Verifica llaves, comas y comillas dobles

`csv sin newline=''`

Causa: Líneas en blanco extra al escribir CSV

Solución: Agrega `newline=""` en `open()`

Hoja de Referencia Rápida

Operación	Código
Leer .txt	<code>with open("f.txt") as f: f.read()</code>
Escribir .txt	<code>with open("f.txt","w") as f: f.write("texto")</code>
Añadir .txt	<code>with open("f.txt","a") as f: f.write("más")</code>
Leer CSV	<code>csv.reader(file) o csv.DictReader(file)</code>
Escribir CSV	<code>csv.writer(f).writerows(data)</code>
Leer JSON	<code>data = json.load(file)</code>
Escribir JSON	<code>file.write(json.dumps(data))</code>



Resumen de la Clase



Archivos .txt: texto plano — `open()`, `read()`, `write()`, `with`



"w" borra todo, "a" añade al final, "r" solo lee — ¡no los confundas!



`with open()` cierra el archivo automáticamente — úsalo siempre



CSV: datos tabulares — `reader()`, `writer()`, `DictReader`, `DictWriter`



JSON: datos estructurados — `json.load()` para leer, `json.dumps()` para escribir



Cada formato tiene su uso: .txt libre, .csv tablas, .json objetos

¡Ahora puedes guardar, leer y compartir datos en cualquier formato! 🚀

¿Preguntas?

*"Los datos sin guardar son como
pensamientos sin escribir: se pierden"*